



Universidade Estadual de Santa Cruz – UESC

**Relatorios de Implementacoes de Metodos da Disciplina
Analise Numerica**

Relatorio de implementacoes realizadas

por Reinan Lopes Argolo

Disciplina Analise Numerica.

Curso Ciencia da Computacao

Semestre 2026.1

Professor Gesil Sampaio Amarante II

Ilheus – BA

2026

Sumário

1	Linguagem escolhida e justificativas	1
2	Regressao por Minimos Quadrados	2
2.1	Regressao linear simples	2
2.1.1	Metodo implementado	2
2.1.2	Exercicio 8.1: regressao linear dos acidentes	2
2.1.3	Exercicio 8.11: modelo de potencia para o PNB	4
2.2	Regressao Polinomial (caso discreto)	5
2.2.1	Metodo implementado	5
2.2.2	Exercicio 8.1: regressao quadratica da taxa de acidentes	6
2.2.3	Exercicio 8.5: coeficiente de placa de orificio	7
2.3	Regressao Polinomial (caso continuo)	8
2.3.1	Metodo implementado	8
2.3.2	Implementacao	9
2.4	Validacao dos resultados de regressao	9
3	Interpolacao Polinomial	10
3.1	Metodo implementado e objetivo numerico	10
3.2	Implementacao em Python	10
3.3	Exercicio 10.2: trajetoria de um projtil	10
3.4	Exercicio 10.6: resistencia de um fio	12
3.5	Exercicio 10.9: circuito com varistor	13
3.6	Validacao dos resultados de interpolacao	14
4	Derivacao e Integracao Numerica	15
4.1	Metodos implementados e objetivo numerico	15
4.2	Implementacao em Python	15
4.3	Exercicio 11.1: eficiencia luminosa de corpo negro	15
4.4	Exercicio 11.6: radiacao emitida por radiador perfeito	17
4.5	Exercicio 11.11: probabilidade na distribuicao normal	18
4.6	Validacao dos resultados de integracao	19
5	Consideracoes finais	20

Lista de Figuras

1	Regressao linear do numero de acidentes no exercicio 8.1.	3
2	Ajuste por modelo de potencia para os PNBs do exercicio 8.11.	5
3	Regressao quadratica da taxa de acidentes no exercicio 8.1.	6
4	Ajuste quadratico para o coeficiente C no exercicio 8.5.	8
5	Trajectoria interpolada do projatil no exercicio 10.2.	11
6	Interpolacao da resistencia do fio no exercicio 10.6.	13
7	Interpolacao da curva do varistor no exercicio 10.9.	14
8	Eficiencia luminosa em funcao da temperatura no exercicio 11.1. . . .	17
9	Radiacao calculada para 2000 K e 6000 K no exercicio 11.6.	18
10	Distribuicao normal e area acumulada ate 5 kg no exercicio 11.11. . . .	19

1 Linguagem escolhida e justificativas

As implementações dos métodos numéricos foram feitas em Python. A escolha da linguagem foi motivada pela legibilidade da sintaxe, pela facilidade de separar o código em funções pequenas e pela disponibilidade de recursos nativos para manipular arquivos de texto e arquivos CSV em UTF-8.

Um cuidado importante do projeto foi não usar bibliotecas prontas para resolver os métodos numéricos. Assim, não foram usadas funções como `numpy.polyfit`, rotinas de integração do `scipy`, modelos de regressão do `sklearn` ou qualquer ferramenta equivalente. As fórmulas de mínimos quadrados, interpolação, derivação, integração, extrapolação de Richardson e quadratura de Gauss foram implementadas diretamente no código.

As bibliotecas usadas tiveram apenas papel de apoio:

- `math`, para funções elementares como exponencial, logaritmo, seno, cosseno e raiz quadrada;
- `csv`, para gravar tabelas de saída em formato CSV;
- `pathlib`, para organizar caminhos de arquivos de forma mais clara;
- `matplotlib`, apenas para gerar gráficos dos resultados já calculados pelos métodos implementados manualmente.

As entradas dos exercícios foram guardadas em arquivos TXT, dentro da pasta de entradas do projeto. As saídas numéricas foram salvas em CSV na pasta de resultados, e as figuras usadas no relatório foram salvas em uma pasta própria para gráficos. Essa separação facilita a repetição dos testes: a execução do script principal de testes regenera os resultados e os gráficos do relatório a partir dos dados de entrada.

2 Regressao por Minimos Quadrados

O capitulo de regressao foi organizado em tres partes, de acordo com os metodos implementados no projeto: regressao linear simples, regressao polinomial no caso discreto e regressao polinomial no caso contínuo. Em todos os casos, a ideia numerica e escolher os coeficientes que tornam minima a soma dos quadrados dos residuos.

Para pontos (x_i, y_i) , o residuo e

$$r_i = y_i - p(x_i),$$

e o criterio minimizado no caso discreto e

$$E = \sum_{i=1}^n r_i^2.$$

A leitura dos dados e feita a partir de arquivos TXT em UTF-8. Cada linha de entrada contem os valores numericos do problema, e linhas iniciadas por # sao ignoradas. Os resultados sao gravados em CSV tambem em UTF-8, usando a funcao auxiliar `salvar_csv`. As figuras do relatorio sao geradas pelo script de testes, sem usar bibliotecas prontas para calcular os ajustes.

2.1 Regressao linear simples

2.1.1 Metodo implementado

Na regressao linear simples, o modelo ajustado tem a forma

$$y = a_0 + a_1 x.$$

Os coeficientes a_0 e a_1 sao obtidos pela minimizacao de

$$E(a_0, a_1) = \sum_{i=1}^n [y_i - (a_0 + a_1 x_i)]^2.$$

Derivando E em relacao aos coeficientes, obtem-se o sistema normal:

$$\begin{cases} na_0 + a_1 \sum x_i = \sum y_i, \\ a_0 \sum x_i + a_1 \sum x_i^2 = \sum x_i y_i. \end{cases}$$

No codigo, esse metodo aparece em `metodos/regressao_linear_mmq.py`. O modulo calcula os coeficientes da reta, o coeficiente de correlacao de Pearson, o coeficiente de determinacao R^2 , os residuos e o desvio padrao dos residuos. Para manter o codigo simples, os somatorios sao feitos diretamente com lacos, sem usar rotinas prontas de regressao.

2.1.2 Exercicio 8.1: regressao linear dos acidentes

Enunciado e objetivo numerico O exercicio 8.1 apresenta dados sobre acidentes em veiculos motorizados no Brasil entre 1980 e 1998. Nesta parte, foi resolvido o item que pede uma regressao linear para o numero de acidentes e uma previsao para o ano 2000.

Metodo empregado Foi usada regressao linear simples por minimos quadrados. Seguindo o enunciado, a variavel independente foi deslocada:

$$x = ano - 1980.$$

Assim, o ano 2000 corresponde a $x = 20$.

Desenvolvimento matematico Para os acidentes em milhares, o modelo usado foi

$$y = a_0 + a_1x.$$

Depois de montar os somatorios do sistema normal, o ajuste obtido foi:

$$y = 8.02159 + 0.362485x.$$

Implementacao Os dados foram lidos do TXT do exercicio. O script principal de testes separa a coluna de acidentes em milhares, chama a rotina de ajuste linear, avalia a reta em $x = 20$, calcula residuos, soma dos quadrados dos residuos e R^2 . A tabela detalhada foi salva em CSV, com valores observados, valores estimados e residuos.

Resultados Para $x = 20$, correspondente ao ano 2000:

$$y(20) = 15.2713.$$

Logo, a previsao foi de aproximadamente 15.2713 milhares de acidentes.

indicador	valor
soma dos residuos ao quadrado	2.123973
R^2	0.938071

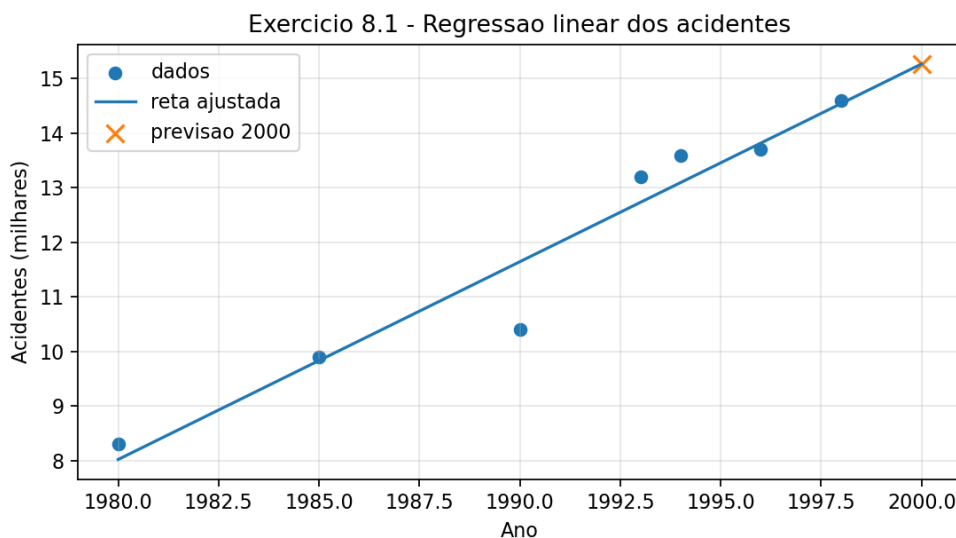


Figura 1: Regressao linear do numero de acidentes no exercicio 8.1.

Interpretacao O grafico mostra uma tendencia geral de crescimento do numero de acidentes, embora alguns pontos intermediarios se afastem da reta. O valor de R^2 proximo de 0.94 indica que o modelo linear explica boa parte da variacao observada na tabela.

2.1.3 Exercicio 8.11: modelo de potencia para o PNB

Enunciado e objetivo numerico O exercicio 8.11 fornece o Produto Nacional Bruto em dolares correntes e em dolares constantes. O enunciado pede que ambas as series sejam aproximadas por uma funcao do tipo:

$$y = ax^b.$$

Depois, o modelo deve ser usado para prever os valores no ano 2000.

Metodo empregado Embora o modelo original seja uma potencia, ele foi resolvido por regressao linear simples apos a transformacao logaritmica:

$$\ln y = \ln a + b \ln x.$$

Assim, o ajuste linear e feito entre $X = \ln x$ e $Y = \ln y$. Como x precisa ser positivo, foi usado $x = ano - 1979$.

Implementacao A funcao de regressao por potencia transforma os pontos por logaritmo, chama o ajuste linear e depois recupera $a = e^{c_0}$. O CSV resumido guarda os parametros dos dois modelos, e a tabela detalhada registra valores observados, valores estimados e residuos.

Resultados Para o PNB em dolares correntes:

$$y = 225.107890x^{0.425689}, \quad y(21) = 822.7053.$$

Para o PNB em dolares constantes:

$$y = 329.451061x^{0.236483}, \quad y(21) = 676.8197.$$

A razao entre as previsoes foi:

$$\frac{822.7053}{676.8197} = 1.21555.$$

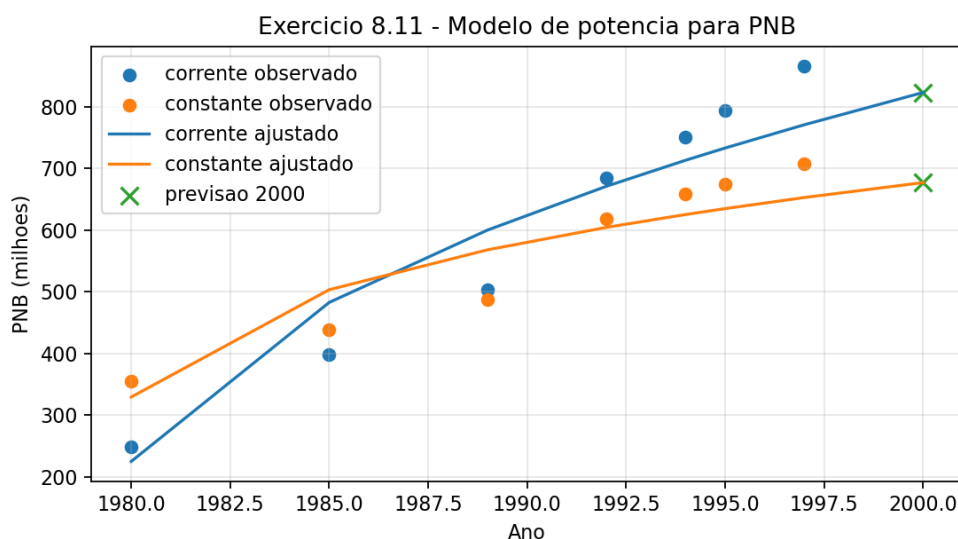


Figura 2: Ajuste por modelo de potencia para os PNBs do exercicio 8.11.

Interpretacao O modelo indica que o PNB em dolares correntes cresce mais rapidamente do que o PNB em dolares constantes. A razao 1.21555 sugere que, para 2000, o valor em dolares correntes seria cerca de 21.55% maior que o valor em dolares constantes. Numericamente, essa diferenca representa o efeito acumulado da inflacao no modelo ajustado.

2.2 Regressao Polinomial (caso discreto)

2.2.1 Metodo implementado

No caso discreto, o objetivo e ajustar um polinomio de grau m a um conjunto finito de pontos experimentais:

$$p(x) = c_0 + c_1x + c_2x^2 + \dots + c_mx^m.$$

O codigo aceita qualquer grau escolhido pelo usuario, desde que haja dados suficientes para montar um sistema coerente.

O sistema normal implementado e:

$$\sum_{j=0}^m c_j \sum_{i=1}^n x_i^{j+k} = \sum_{i=1}^n y_i x_i^k, \quad k = 0, 1, \dots, m.$$

Esse sistema e montado manualmente com somatorios e resolvido pela eliminacao de Gauss do proprio projeto.

Os codigos principais usados nesta parte sao o modulo de ajuste polinomial discreto, o resolvidor de sistemas lineares e o modulo auxiliar de polinomios. O primeiro monta as equacoes normais, o segundo resolve o sistema linear, e o terceiro avalia e formata o polinomio encontrado.

2.2.2 Exercício 8.1: regressão quadrática da taxa de acidentes

Enunciado e objetivo numérico O item polinomial do exercício 8.1 pede uma regressão quadrática para a taxa de acidentes por 10000 veículos e uma previsão para o ano 2000.

Método empregado Foi usado mínimos quadrados polinomial discreto de grau 2, com a mesma variável deslocada $x = ano - 1980$.

Desenvolvimento matemático O modelo usado foi

$$y = a_0 + a_1x + a_2x^2.$$

Os coeficientes foram obtidos pelas equações normais do caso discreto.

Implementação O script principal de testes lê a coluna de taxa de acidentes, monta o sistema normal de grau 2, resolve os coeficientes, avalia o polinômio em $x = 20$ e salva uma tabela detalhada com resíduos.

Resultados O polinômio ajustado foi:

$$y = 1.6985 - 0.0368563x + 0.00115007x^2.$$

No ano 2000:

$$y(20) = 1.42140.$$

indicador	valor
soma dos resíduos ao quadrado	0.004692
R^2	0.939921

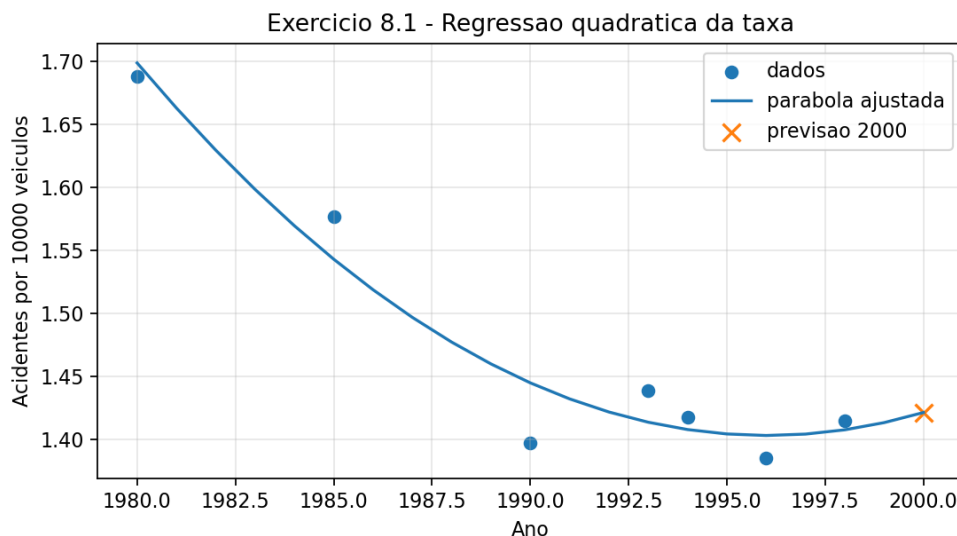


Figura 3: Regressão quadrática da taxa de acidentes no exercício 8.1.

Interpretacao A parabola captura uma queda inicial da taxa de acidentes e uma leve retomada ao final da tabela. O valor de R^2 ficou muito proximo do obtido na parte linear, mas agora o modelo descreve uma tendencia curva, adequada ao comportamento da taxa.

2.2.3 Exercicio 8.5: coeficiente de placa de orificio

Enunciado e objetivo numerico O exercicio 8.5 fornece valores experimentais do coeficiente C em funcao da razao $x = A/A_1$. O objetivo e ajustar os pontos por

$$C(x) = a_0 + a_1x + a_2x^2.$$

Metodo empregado Foi usado minimos quadrados polinomial discreto de grau 2, pois ha mais pontos experimentais do que coeficientes. Assim, em vez de interpolar todos os dados, o programa encontra a parabola que minimiza a soma dos quadrados dos residuos.

Desenvolvimento matematico O sistema normal montado pelo codigo foi:

$$\begin{bmatrix} 10 & 5.5 & 3.85 \\ 5.5 & 3.85 & 3.025 \\ 3.85 & 3.025 & 2.5333 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 7.4 \\ 4.391 \\ 3.2319 \end{bmatrix}.$$

Resolvendo esse sistema por eliminacao de Gauss, obtem-se os coeficientes do polinomio ajustado.

Implementacao Os dados foram lidos do TXT do exercicio. A funcao de ajuste polinomial monta as equacoes normais e chama o resolvedor de Gauss. O sistema normal foi salvo em um CSV proprio, e a tabela completa de comparacao foi salva no CSV principal do exercicio.

Resultados O polinomio obtido foi:

$$C(x) = 0.650167 - 0.231742x + 0.564394x^2.$$

x	C observado	C ajustado	residuo
0.10	0.62	0.632636	-0.012636
0.50	0.68	0.675394	0.004606
0.80	0.81	0.825985	-0.015985
1.00	1.00	0.982818	0.017182

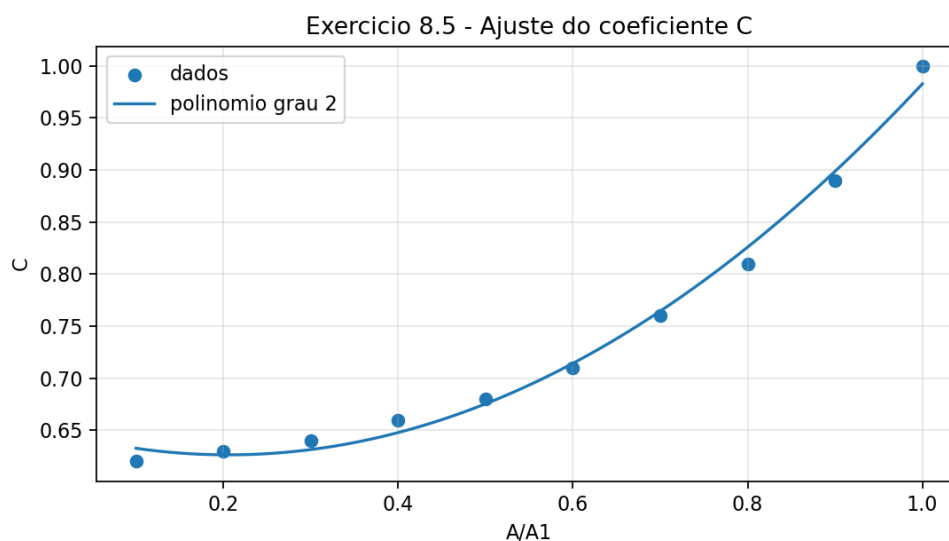


Figura 4: Ajuste quadrático para o coeficiente C no exercício 8.5.

Interpretacao O ajuste acompanha a tendência crescente dos dados. Os maiores resíduos em módulo aparecem próximos das extremidades, principalmente em $x = 1$, onde o valor tabelado é 1.00 e o valor ajustado é 0.982818. Ainda assim, os resíduos são pequenos para a escala do problema, e o gráfico mostra uma curva coerente com os pontos experimentais.

2.3 Regressão Polinomial (caso contínuo)

2.3.1 Método implementado

No caso contínuo, o objetivo é aproximar uma função $f(x)$, definida em um intervalo $[a, b]$, por um polinômio

$$p(x) = c_0 + c_1x + c_2x^2 + \dots + c_mx^m.$$

Agora, em vez de minimizar uma soma finita de resíduos, minimiza-se a integral do erro quadrático:

$$E = \int_a^b [f(x) - p(x)]^2 dx.$$

As equações normais deixam de usar somatórios e passam a usar integrais:

$$\sum_{j=0}^m c_j \int_a^b x^{j+k} dx = \int_a^b f(x) x^k dx, \quad k = 0, 1, \dots, m.$$

No código, as integrais $\int_a^b x^{j+k} dx$ são calculadas pela fórmula exata da potência. Já as integrais que envolvem $f(x)$ são aproximadas por Simpson composto, também implementado manualmente no projeto.

2.3.2 Implementacao

O metodo continuo esta em `metodos/mmq_continuo_polynomial.py`. O arquivo de entrada informa a funcao, os limites do intervalo, o grau do polinomio e a quantidade de subintervalos usada na regra de Simpson. A funcao principal le esses parametros, monta o sistema normal continuo, resolve os coeficientes por eliminacao de Gauss e salva um CSV comparando $f(x)$, $p(x)$ e o erro em pontos igualmente espacados do intervalo.

Observacao sobre os exercicios do livro Entre os exercicios obrigatorios indicados para o relatorio, os problemas do capitulo 8 usados como teste envolvem dados discretos ou modelos linearizados. Por isso, esta subsecao registra a implementacao do caso continuo, mas nao atribui a ela um exercicio do livro que nao tenha sido pedido no enunciado.

2.4 Validacao dos resultados de regressao

Os graficos confirmam que os ajustes seguem a tendencia principal dos dados. Em 8.1, tanto a regressao linear dos acidentes quanto a regressao quadratica da taxa apresentaram R^2 proximo de 0.94, indicando boa capacidade explicativa para uma tabela pequena. Em 8.5, os residuos sao pequenos e alternam sinais, comportamento esperado em um ajuste por minimos quadrados. Em 8.11, a transformacao logaritmica permitiu tratar o modelo de potencia com as mesmas ferramentas da regressao linear simples, sem usar rotinas prontas de ajuste. O caso continuo tambem permanece implementado no projeto, separado do caso discreto porque sua formulacao usa integrais em vez de somatorios.

3 Interpolacao Polinomial

3.1 Metodo implementado e objetivo numerico

A interpolacao polinomial foi usada para construir um polinomio que passa exatamente por um conjunto de pontos conhecidos. Diferentemente dos minimos quadrados, aqui nao se busca reduzir residuos: se os pontos sao distintos, o polinomio interpolador satisfaz

$$P(x_i) = y_i, \quad i = 0, 1, \dots, n.$$

Foram implementadas duas formas equivalentes do polinomio interpolador: Lagrange e Newton por diferencas divididas. A forma de Lagrange e direta e boa para demonstrar a construcao teorica. A forma de Newton e mais pratica quando se deseja reaproveitar uma tabela de diferencas divididas.

3.2 Implementacao em Python

Os arquivos principais sao:

- `metodos/interpolacao_lagrange.py`, que monta as bases $L_i(x)$, multiplica polinomios e soma os termos $y_i L_i(x)$;
- `metodos/interpolacao_newton.py`, que calcula diferencas divididas e avalia o polinomio pela forma aninhada de Newton;
- `metodos/polinomios.py`, que avalia, soma, multiplica e formata polinomios;
- `testes_livro_neide.py`, que seleciona os dados dos exercicios, escolhe pontos proximos quando necessario e gera os CSVs.

Na forma de Lagrange, cada base e calculada por

$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j},$$

e o polinomio final e

$$P(x) = \sum_{i=0}^n y_i L_i(x).$$

Na forma de Newton, o polinomio e escrito como

$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + \dots + f[x_0, \dots, x_n](x - x_0) \dots (x - x_{n-1}).$$

O codigo tambem converte o polinomio para potencias de x , pois essa forma e mais simples de apresentar no relatorio.

3.3 Exercicio 10.2: trajetoria de um projtil

Enunciado e objetivo numerico O exercicio informa que um projtil parte da origem, passa pelo ponto $(10, 6)$ e atinge uma barreira em $(20, 4)$. Com esses tres pontos, o objetivo e interpolar a trajetoria, determinar o angulo de lancamento ψ , a velocidade inicial v_0 , e calcular a altitude em $x = 5$ m.

Metodo empregado Foi usada interpolacao por Lagrange, pois os tres pontos determinam uma unica parabola. Como a trajetoria fisica de um projetal sem resistencia do ar tambem e quadratica em x , o polinomio interpolador tem interpretacao direta.

Desenvolvimento matematico Com os pontos $(0, 0)$, $(10, 6)$ e $(20, 4)$, o polinomio obtido foi:

$$P(x) = x - 0.04x^2.$$

Comparando com a equacao teorica

$$y = x \tan \psi - \frac{1}{2} g \frac{x^2}{v_0^2 \cos^2 \psi},$$

temos:

$$\tan \psi = 1 \Rightarrow \psi = 45^\circ.$$

Como o coeficiente de x^2 e -0.04 , usando $g = 9.86$, obtem-se:

$$v_0 = \sqrt{\frac{-g(1 + \tan^2 \psi)}{2(-0.04)}} = 15.7003 \text{ m/s}.$$

Implementacao Os dados foram lidos do TXT do exercicio. A rotina de Lagrange construiu o polinomio, e a rotina de avaliacao foi usada para calcular $P(5)$. O resumo numerico fica no CSV do exercicio.

Resultados A altitude em $x = 5$ foi:

$$P(5) = 5 - 0.04(25) = 4.0 \text{ m}.$$

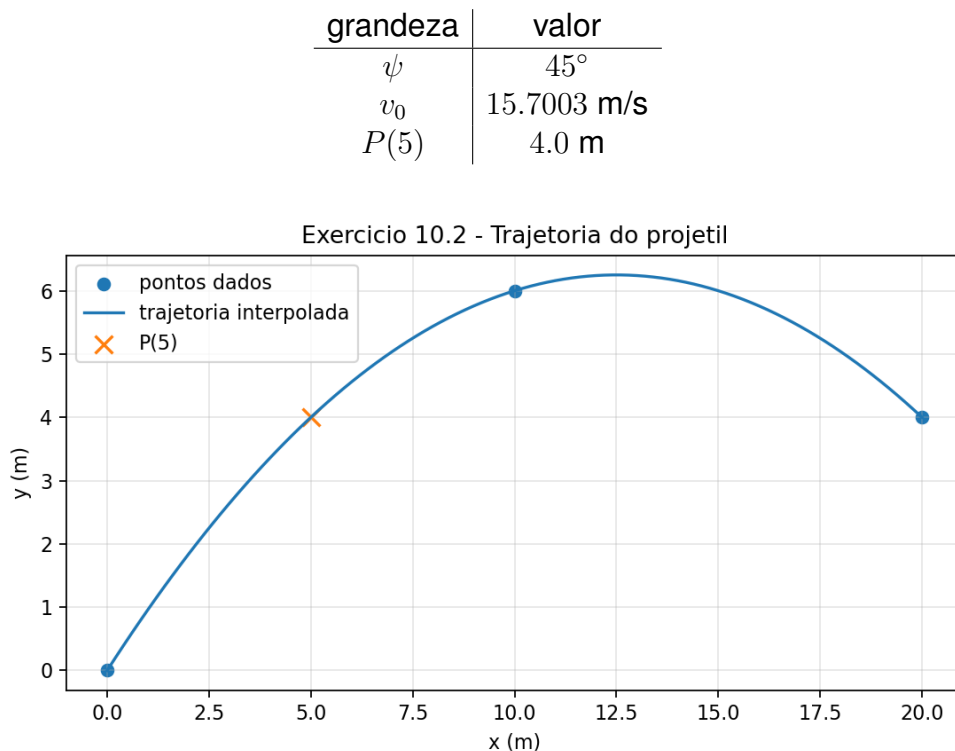


Figura 5: Trajetoria interpolada do projetal no exercicio 10.2.

Interpretacao O grafico mostra a parabola passando exatamente pelos tres pontos informados no enunciado. O angulo de 45° surge porque o termo linear do polinomio e igual a 1, isto e, $\tan \psi = 1$. A altitude em 5 m fica abaixo da altitude em 10 m, mas ainda em subida, coerente com o formato da trajetoria.

3.4 Exercicio 10.6: resistencia de um fio

Enunciado e objetivo numerico O exercicio fornece resistencias medidas para varios comprimentos de um fio e pede estimativas para 1730 m e 3200 m usando parabolas de grau 2 e grau 3.

Metodo empregado Foi usada interpolacao de Newton por diferencas divididas. Para cada valor a estimar, o programa escolheu os pontos mais proximos, pois a interpolacao tende a ser mais confiavel quando os nos estao perto do ponto consultado.

Desenvolvimento matematico Para $x = 1730$, os pontos usados no grau 2 foram 1000, 1500 e 2000. No grau 3, foi acrescentado o ponto 2500. Para $x = 3200$, foram usados os pontos 2500, 3000, 3500 e, no grau 3, tambem 4000.

Um exemplo de tabela de diferencas divididas para 1730 m e grau 2 e:

x	$f[x]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$
1000	5.48	0.00484	$1.36 \cdot 10^{-6}$
1500	7.90	0.00620	
2000	11.00		

Implementacao A tabela de entrada esta no TXT do exercicio. A rotina de diferencas divididas calcula os coeficientes de Newton, e a conversao para potencias gera uma forma expandida do polinomio. Os pontos escolhidos e as diferencas divididas foram salvos em CSVs separados para facilitar a verificacao.

Resultados

comprimento	grau	resistencia aproximada
1730	2	9.241544
1730	3	9.292921
3200	2	17.796800
3200	3	17.899840

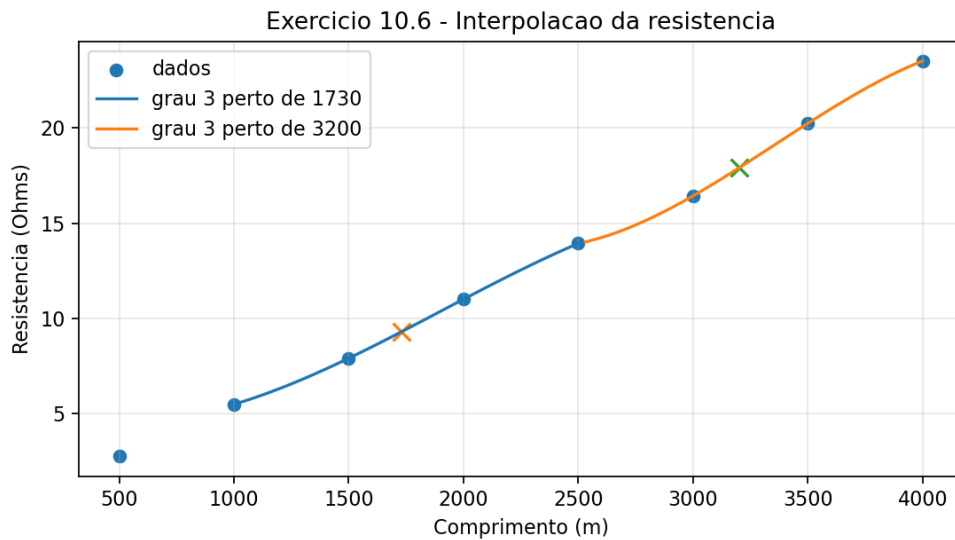


Figura 6: Interpolacao da resistencia do fio no exercicio 10.6.

Interpretacao As estimativas de grau 2 e grau 3 ficaram proximas, mas nao identicas. Isso e esperado: ao usar mais um ponto, o polinomio passa a incorporar informacao de uma regiao um pouco maior da tabela. A coerencia entre os dois valores sugere que os resultados sao estaveis para os pontos escolhidos.

3.5 Exercicio 10.9: circuito com varistor

Enunciado e objetivo numerico O exercicio apresenta uma tabela de corrente I em funcao da tensao E_2 em um varistor. No item (a), pede-se a corrente para $E_2 = 2.3$, e depois as tensoes E_1 e E no circuito com $R_1 = 10\Omega$. No item (b), pede-se uma interpolacao inversa de grau 2 para determinar E_2 quando $E_1 = 10$ V.

Metodo empregado No item (a), foi usada interpolacao direta por Newton sobre todos os pontos da tabela. No item (b), foi feita interpolacao inversa: a corrente passou a ser a variavel independente e E_2 passou a ser a variavel dependente. Como

$$E_1 = R_1 I,$$

quando $E_1 = 10$ e $R_1 = 10$, temos $I = 1$.

Implementacao Os dados foram lidos do TXT do exercicio. A avaliacao do polinomio de Newton calculou $I(2.3)$. Para o item (b), foram escolhidos os tres pontos mais proximos de $I = 1$, salvos em um CSV especifico da interpolacao inversa.

Resultados Para $E_2 = 2.3$:

$$I = 0.810152 \text{ A.}$$

Assim:

$$E_1 = 10I = 8.10152 \text{ V}, \quad E = E_1 + E_2 = 10.40152 \text{ V.}$$

Na interpolacao inversa, para $I = 1$, o resultado foi:

$$E_2 = 2.46417 \text{ V.}$$

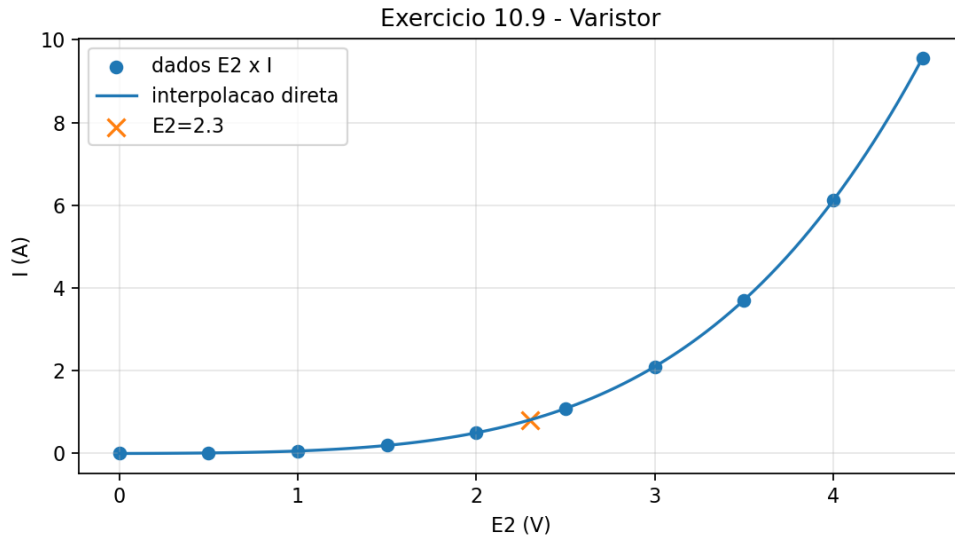


Figura 7: Interpolacao da curva do varistor no exercicio 10.9.

Interpretacao A curva do varistor cresce de forma nao linear, confirmando que a relacao entre corrente e tensao nao e simplesmente ohmica. A interpolacao direta fornece a corrente para uma tensao intermediaria, enquanto a interpolacao inversa permite resolver o problema contrario: encontrar a tensao associada a uma corrente determinada.

3.6 Validacao dos resultados de interpolacao

No exercicio 10.2, a parabola passa exatamente pelos pontos do enunciado e possui interpretacao fisica coerente. No exercicio 10.6, os resultados de grau 2 e 3 sao proximos, indicando estabilidade local da interpolacao. No exercicio 10.9, o grafico confirma a curvatura da relacao $E_2 \times I$, justificando o uso de um polinomio em vez de uma reta.

4 Derivacao e Integracao Numerica

4.1 Metodos implementados e objetivo numerico

Esta parte do projeto reúne formulas de derivacao numerica e regras de integracao. As derivadas numericas aproximam $f'(x)$ e $f''(x)$ a partir de valores da funcao em pontos proximos. As regras de integracao aproximam integrais definidas quando a primitiva nao e conveniente ou quando os dados estao em forma numerica.

Foram implementadas:

- diferencas progressiva, regressiva e central para a primeira derivada;
- diferenca central para a segunda derivada;
- extrapolacao de Richardson;
- trapezio simples e composto;
- Simpson 1/3 simples e composto;
- Simpson 3/8 simples e composto;
- quadratura de Gauss-Legendre com 2, 3 e 4 pontos.

4.2 Implementacao em Python

As funcoes de derivacao estao no modulo de derivadas numericas, e as regras de integracao estao no modulo de integracao numerica. A funcao de refinamento, definida no script de testes do livro, usa Simpson 1/3 composto e dobra o numero de subintervalos ate que a variacao relativa entre duas aproximacoes consecutivas seja menor que a tolerancia do enunciado.

Para a regra de Simpson 1/3 composta, usada nos exercicios aplicados, o codigo calcula:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{i \text{ impar}} f(x_i) + 2 \sum_{i \text{ par}} f(x_i) + f(x_n) \right],$$

com n par e $h = (b - a)/n$.

A quadratura de Gauss-Legendre foi implementada manualmente com raizes e pesos tabelados para 2, 3 e 4 pontos. A transformacao de intervalo usada foi:

$$x = \frac{a+b}{2} + \frac{b-a}{2}t.$$

4.3 Exercicio 11.1: eficiencia luminosa de corpo negro

Enunciado e objetivo numerico O exercicio 11.1 pede a eficiencia luminosa de um corpo negro, definida por uma integral da equacao de Planck no intervalo visivel de comprimento de onda. A expressao usada foi:

$$EFF = \frac{64.77}{T^4} \int_{4 \cdot 10^{-5}}^{7 \cdot 10^{-5}} \frac{dx}{x^5 (e^{1.432/(Tx)} - 1)}.$$

O enunciado pede erro relativo menor que 10^{-5} .

Metodo empregado Foi usada a regra de Simpson 1/3 composta com refinamento automatico. A cada passo, o numero de subintervalos e dobrado. A parada ocorre quando

$$\frac{|I_{novo} - I_{antigo}|}{\max(1, |I_{novo}|)} < 10^{-5}.$$

Implementacao Os parametros foram lidos do TXT do exercicio. O script gerou um CSV principal com as eficiencias e outro CSV com o historico de refinamento. Esse historico registra, para cada temperatura, a quantidade de subintervalos usada e o erro relativo estimado.

Resultados Para o conjunto com incremento de 250 K:

$T(K)$	eficiencia	subintervalos
2000	0.805807	32
2250	1.828785	32
2500	3.426141	32
2750	5.595158	32
3000	8.255324	16

Para o conjunto com incremento de 200 K, as eficiencias foram:

$T(K)$	eficiencia
2000	0.805807
2200	1.579765
2400	2.716444
2600	4.228195
2800	6.091532
3000	8.255324

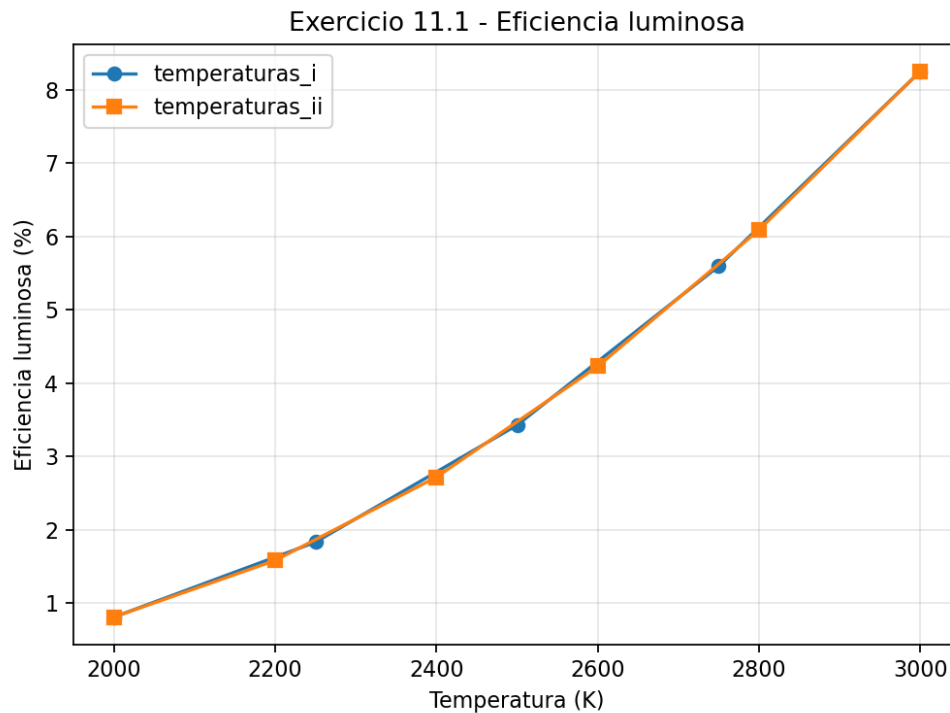


Figura 8: Eficiência luminosa em função da temperatura no exercício 11.1.

Interpretacao A eficiência cresce com a temperatura, comportamento fisicamente coerente: ao aumentar T , uma parcela maior da radiação emitida se desloca para a faixa visível. O refinamento convergiu com poucos subintervalos, e os erros relativos registrados no CSV ficaram abaixo de 10^{-5} .

4.4 Exercício 11.6: radiação emitida por radiador perfeito

Enunciado e objetivo numerico O exercício 11.6 pede o cálculo da radiação emitida por unidade de área e tempo:

$$Q = \int_{\lambda_1}^{\lambda_2} \frac{2\pi hc^2}{\lambda^5 (e^{hc/(k\lambda T)} - 1)} d\lambda,$$

com $\lambda_1 = 3.933666 \cdot 10^{-5}$ cm, $\lambda_2 = 5.895923 \cdot 10^{-5}$ cm e temperaturas de 2000 K e 6000 K.

Metodo empregado Também foi usada Simpson 1/3 composta com refinamento automático até erro relativo menor que 10^{-5} . Essa escolha é adequada porque a função é contínua no intervalo informado e pode ser avaliada diretamente.

Implementacao Os dados foram lidos do TXT do exercício. O código usa as constantes $h = 6.6256 \cdot 10^{-27}$, $c = 2.99793 \cdot 10^{10}$ e $k = 1.38054 \cdot 10^{-16}$. Os resultados finais e o refinamento foram salvos em CSV.

Resultados

$T(K)$	Q	subintervalos	erro relativo
2000	$1.631928752 \cdot 10^6$	16	$7.65 \cdot 10^{-6}$
6000	$1.900530912 \cdot 10^{10}$	8	$9.90 \cdot 10^{-7}$

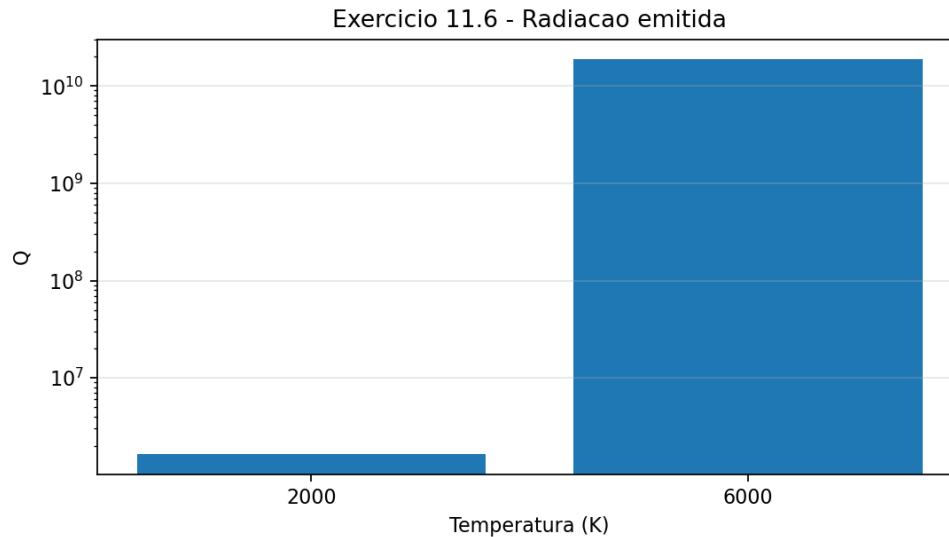


Figura 9: Radiação calculada para 2000 K e 6000 K no exercício 11.6.

Interpretação O valor de Q para 6000 K é várias ordens de grandeza maior que para 2000 K, o que é coerente com a forte dependência da radiação em relação à temperatura. O gráfico foi colocado em escala logarítmica para tornar visível essa diferença.

4.5 Exercício 11.11: probabilidade na distribuição normal

Enunciado e objetivo numérico O exercício 11.11 trata do peso de pacotes de açúcar de 5 kg. A média da amostra é $\bar{x} = 4.991$ kg e o desvio padrão é $S = 0.005$ kg. O objetivo é calcular a probabilidade acumulada até $x = 5$:

$$F(5) = \int_{-\infty}^5 \frac{1}{S\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\bar{x}}{S}\right)^2} dx.$$

Método empregado Como a distribuição normal é simétrica em torno da média, foi usado:

$$F(5) = 0.5 + \int_{\bar{x}}^5 f(x) dx.$$

A integral de $\bar{x}$ até 5 foi calculada por Simpson composto com refinamento por tolerância.

Implementação Os parâmetros foram lidos do TXT do exercício. O resultado final e o histórico de refinamento foram salvos em CSV.

Resultados O valor encontrado foi:

$$F(5) = 0.964070.$$

Logo:

$$P(X \leq 5) = 96.407\%, \quad P(X > 5) = 3.593\%.$$

Em uma amostra de 100 pacotes, isso corresponde a aproximadamente 96.41 pacotes com peso menor ou igual a 5 kg.

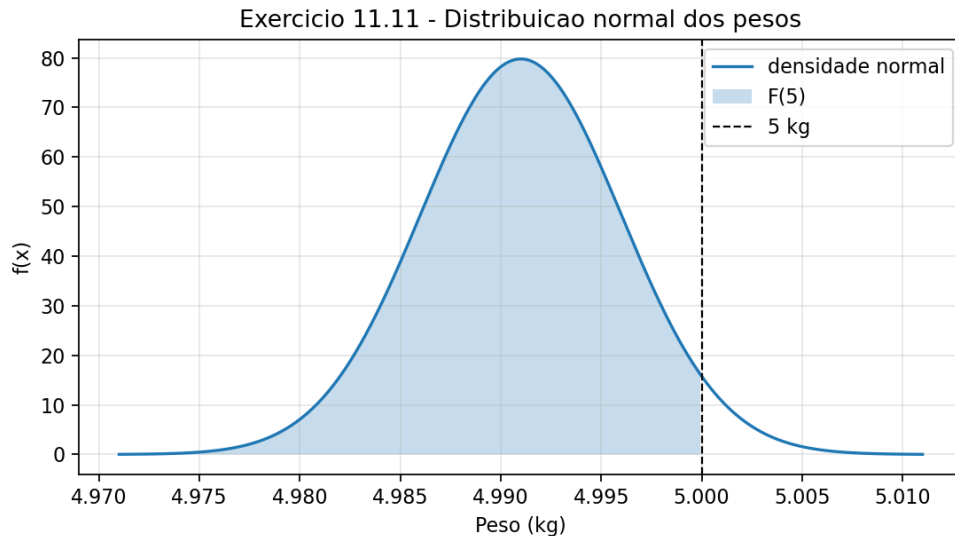


Figura 10: Distribuicao normal e area acumulada ate 5 kg no exercicio 11.11.

Interpretacao Como a media esta abaixo de 5 kg, a area ate 5 kg e maior que 50%. O resultado de aproximadamente 96.4% indica que quase todos os pacotes devem pesar no maximo 5 kg, dentro do modelo normal assumido pelo enunciado. A cauda acima de 5 kg e pequena, cerca de 3.6%.

4.6 Validacao dos resultados de integracao

Nos tres exercicios, o processo de refinamento por tolerancia convergiu. Os graficos confirmam a coerencia dos resultados: a eficiencia luminosa cresce com a temperatura, a radiacao emitida aumenta fortemente quando a temperatura passa de 2000 K para 6000 K, e a area acumulada da normal fica majoritariamente a esquerda de 5 kg porque a media e 4.991 kg.

5 Consideracoes finais

Ao longo das implementacoes, a prioridade sera manter os algoritmos numericos visiveis no codigo, com entradas e saidas padronizadas em UTF-8. Essa organizacao facilita a inclusao de novos metodos e a reproducao dos testes descritos no relatorio.

Referências

- [1] FRANCO, Neide Bertoldi. *Calculo Numerico*. Sao Paulo: Pearson, 2006.